

Fresher Track — Lean Study Handbook

This is the trimmed companion to the Master Handbook. It contains only what's needed to: (1) clear DSA rounds, (2) build all ten projects, and (3) explain them and answer fresher-level interview questions confidently. Depth topics (distributed tracing, bulkhead patterns, JVM internals, HLD at scale) live in the Master Handbook — come back to that once you're past entry-level rounds, not before.

Section 1: Your Projects, By Role

Role	Project 1	Project 2	Project 3
Frontend Dev	♀ CheckMate Analyze	👛 Trajectory (UI)	📖 StudyLink
Java Backend / Full Stack	🦅 Phoenix	👛 Trajectory	💡 Conclave
Python Backend / Full Stack	🔧 Foundry	🔄 Synod	📖 StudyLink
AI Engineer	🦅 Phoenix	🔧 Foundry	🛡️ Vigil
QA Engineer	🔍 Aegis	📊 Pulse	🔒 Warden

Build order: Trajectory → Phoenix → (Conclave / Foundry in parallel) → Synod / StudyLink → Vigil → Aegis / Pulse / Warden. The QA trio needs Trajectory and Phoenix actually running before there's anything real to test.

Section 2: DSA — Non-Negotiable, Runs the Whole Time

This is the track that decides whether you pass the screening round at all, independent of how good the projects are. Keep running your own custom DSA plan alongside everything below — this section is just the essentials, not a replacement for it.

- **Data structures to know cold:** Arrays, Strings, HashMaps, Linked Lists, Stacks, Queues, Trees/BSTs, Heaps, Graphs (adjacency list, BFS/DFS).
- **Patterns to recognize instantly:** Two Pointers / Sliding Window, Binary Search (including "on the answer space"), Recursion & Backtracking, Dynamic Programming (1D and 2D first), BFS/DFS traversal, Top-K via Heaps.

- **Practice discipline:** one pattern at a time (5-7 days before switching), timed solving (20-25 min, no notes) once a pattern feels learned, explain your approach out loud every time, track fluency over problem count.
- **Weak spot flagged earlier:** Trees/Graphs — revisit on a schedule, don't do them once and move on.

Section 3: Project Build Guides

Each project lists what to learn to build it, how to actually deploy it, what to say to explain it, and the hands-on skill specific to that project that no AI coding agent can do for you.

👤 CheckMate Analyze

- **Stack:** React, TypeScript, Stockfish (WASM), Tailwind, Vite
- **What it is:** Client-side chess analysis — no backend, engine runs entirely in-browser.
- **To build it:**
 - React basics: components, props, hooks (`useState` , `useEffect` , `useRef`)
 - TypeScript basics: types, interfaces
 - `chess.js` for move validation/PGN parsing, `react-chessboard` for the board UI
 - Web Workers — run Stockfish off the main thread so the UI doesn't freeze
 - WebAssembly, at a "what it is" level — compiled code running fast in-browser
- **To deploy it:**
 - Push to GitHub, connect the repo to Vercel — auto-deploys on every push
 - No server, no database, no environment variables to manage — the simplest deployment story in the whole portfolio
 - No separate CI/CD pipeline needed — Vercel's build-and-deploy is built in, triggered by the same push
 - Know why this is possible here specifically: nothing stateful to run, just static files and client-side code
- **To explain it:**
 - The pipeline: PGN → `chess.js` parses moves → FEN sent to Stockfish → evaluation returned → UI updates
 - Why Web Workers exist — without one, the engine would freeze the page
 - Why client-side only was the right call — no server cost, works offline; trade-off is no cross-device history

- **Hands-on skill:**
 - Browser DevTools — Console for JS errors, Network tab for any fetch calls
 - Debug a WASM load failure at least once

Trajectory

- **Stack:** Spring Boot, React, PostgreSQL, Spring AI
- **What it is:** Job application tracker with a status timeline, resume versioning, and AI-powered auto-fill from job descriptions.
- **To build it:**
 - Spring Boot basics: Controllers, Dependency Injection, `@RestControllerAdvice`
 - Spring Data JPA: entity relationships, saving/retrieving data
 - REST fundamentals: HTTP verbs, status codes (200/201/400/404), statelessness
 - JWT authentication basics
 - React + TanStack Query for fetching/caching the applications list
 - A status state machine: Applied → OA → Interview → Offer/Rejected, with a history table
 - Spring AI `ChatClient` — calling an LLM and parsing its response into your data model
- **To deploy it:**
 - Locally: everything runs via Docker Compose — Postgres and the Spring Boot app
 - Production: React frontend deploys to Vercel (same simplicity as CheckMate Analyze)
 - Production: Spring Boot backend deploys to AWS — containerized, run on ECS + Fargate
 - RDS as the managed Postgres, S3 for resume storage
 - CI/CD: GitHub Actions builds the Docker image on every push, pushes it to AWS ECR, then updates the ECS service to run the new image
 - Frontend on Vercel talks to the AWS-hosted API over HTTPS
 - Know this split by name — static frontend on Vercel + stateful backend on AWS is a very common real-world pattern
- **To explain it:**
 - Why a separate status-history table exists, not just a current-status column
 - The end-to-end AI flow: paste a JD → API call → Spring AI parses it → auto-fills the form → user confirms before saving
 - What `@RestControllerAdvice` buys you over try/catch everywhere

- **Hands-on skill:**

- OAuth app registration (Google console — redirect URI, client ID/secret) if you add social login
- Run a real Flyway migration and see what happens when two migrations conflict
- Click through the actual AWS console once to create the RDS instance and S3 bucket, not just see them in a YAML file

Phoenix

- **Stack:** Spring Boot (API layer) + Python/LangGraph (AI engine)

- **What it is:** A RAG system that answers questions about documents, and knows when to say "I'm not sure" instead of guessing.

- **To build it:**

- Embeddings — numeric representation of text meaning; similar text ends up close together
- Vector stores (`pgvector` or `FAISS`) — storing and searching embeddings
- Chunking — splitting documents small enough to embed without losing context
- Hybrid search — combine vector search with keyword search, since vector search alone misses exact terms (an error code, a config key)
- Confidence scoring — simplest version: low similarity score means don't trust the result
- A basic REST contract between two services (Spring Boot calling the Python service)

- **To deploy it:**

- Locally: Docker Compose runs both services (Spring Boot + Python AI engine) on one network, reachable by service name
- Production: both services deploy as separate containers on AWS ECS + Fargate
- CI/CD: GitHub Actions builds both service images on push, pushes each to ECR, and updates both ECS services independently
- This is the clearest example in the portfolio of why containerization matters — two services, two languages, no shared server/runtime needed

- **To explain it:**

- Why hybrid search beats pure vector search, with a concrete example
- The self-healing idea in one sentence: low confidence triggers a clarifying question instead of a guess

- Why it's two services, not one — Java handles auth/API/storage, Python handles the AI-specific work
- **Hands-on skill:**
 - Docker Compose networking — debug why one container can't reach another on the same network
 - `docker exec -it <container> /bin/sh` to inspect what's actually happening inside a running container

Conclave

- **Stack:** Spring Boot, WebSockets
- **What it is:** A shared chat room where you and multiple LLMs (Claude, GPT, Gemini) talk in one conversation — no manual copy-pasting between tabs.
- **To build it:**
 - Spring Boot WebSockets — real-time bidirectional communication
 - Calling multiple provider APIs (Anthropic, OpenAI, Gemini) and noticing they each expect a different request/response shape
 - The Adapter Pattern — one canonical internal message format, translated per-provider on the way in and out
 - Basic rate limiting — since you're paying for three APIs at once
- **To deploy it:**
 - Same pattern as Trajectory — frontend on Vercel, Spring Boot backend on AWS ECS + Fargate
 - CI/CD: same GitHub Actions → ECR → ECS pipeline as Trajectory
 - Know this specific consideration: WebSocket connections need "sticky" routing if scaled to multiple backend instances (a user's connection has to keep hitting the same instance)
 - Worth being able to name this even without fully implementing it at this stage
- **To explain it:**
 - Walk through the adapter layer with a concrete example of a shape mismatch between two providers
 - Why turn-taking is user-directed (@-mention) rather than automated in v1, and what would change to automate it

- Frame this honestly as integration/systems engineering, not AI engineering — say that distinction out loud
- **Hands-on skill:**
 - API key provisioning and quota-dashboard checking across three separate providers simultaneously — a real, easy-to-forget cost management skill

Foundry

- **Stack:** Django, Celery, Redis, LangGraph
- **What it is:** Three AI personas (Investor, PM, Tech Lead) debate and produce a startup plan together, remembering their own past decisions so they don't contradict themselves later.
- **To build it:**
 - Django basics: models, migrations, DRF serializers/viewsets
 - Why Celery exists — Django's request cycle is synchronous, so a slow AI generation would time out a normal request; Celery runs it in the background instead
 - A job status state machine: queued → generating → ready
 - Basic LangGraph — modeling which agent runs next as a graph, not one big function
 - The "decision log" idea — storing why a choice was made, not the raw generated text, so it can be referenced if a section gets regenerated later
- **To deploy it:**
 - Locally: Docker Compose runs Django, the Celery worker, and Redis as three separate containers
 - Notice the Celery worker is a genuinely separate running process from the web server, not a thread inside it
 - Production: Django app and Celery worker each run as their own service on AWS ECS + Fargate
 - Production: Redis moves to a managed ElastiCache instance instead of a self-run container
 - CI/CD: GitHub Actions builds and pushes both the Django app image and the Celery worker image to ECR, then updates both ECS services
- **To explain it:**
 - Why Celery/Redis specifically — without it, the request hangs or times out mid-generation

- What "decision memory" means and how it differs from Phoenix's RAG — memory of the system's own past decisions, not retrieval from outside documents
- **Hands-on skill:**
 - Watch a Celery worker actually pick up and process a queued job
 - Kill a stuck worker process and understand why the job got orphaned



Synod

- **Stack:** Django (Python port of Conclave's real-time layer)
- **What it is:** The same problem as Conclave, deliberately rebuilt in Python to compare the two approaches directly.
- **To build it:**
 - Django Channels + ASGI — what makes Django support WebSockets, since its default mode doesn't
 - Reimplementing the same adapter pattern in Python instead of Java
- **To deploy it:**
 - Same shape as Foundry — Docker Compose locally, ECS + Fargate in production
 - CI/CD: same GitHub Actions → ECR → ECS pipeline as Foundry
 - One added wrinkle: Channels needs an ASGI server (Daphne or Uvicorn) instead of Django's default WSGI server — a small but real deployment configuration difference from a normal Django app
- **To explain it:**
 - Have 2-3 specific differences ready from actually porting it — not "Python is different from Java" in the abstract, but a real friction point (e.g., Channels needing explicit ASGI setup vs. Spring's more native WebSocket handling)
 - Be ready to say why this project exists at all — the comparison is the deliverable
- **Hands-on skill:**
 - Debug an ASGI server misconfiguration — a genuinely different class of error than Spring's WebSocket setup, worth hitting once rather than just reading about it



StudyLink

- **Stack:** Django, React
- **What it is:** A study-resource vault (notes/PDFs) plus an OLX-style local giveaway marketplace for old textbooks — local pickup only, no shipping or payments.

- **To build it:**
 - Django models + DRF for both modules: Resource (vault) and Listing (marketplace)
 - File upload handling — storing and retrieving PDFs/images
 - Search/filter query design across subject/course tags
 - A listing state machine: Available → Requested → Given Away (with a fallback path back to Available)
 - Object-level permissions — only the listing's owner can manage it
- **To deploy it:**
 - Frontend on Vercel, Django backend on AWS ECS + Fargate
 - CI/CD: GitHub Actions → ECR → ECS, same pipeline pattern as the other Django/Spring Boot backends
 - Most important detail: uploaded PDFs/images cannot live inside the container itself (containers are ephemeral — anything on local disk disappears on redeploy)
 - Route file uploads to S3 from day one, not as an afterthought
- **To explain it:**
 - Walk through the listing state machine and what happens at each transition
 - Explain the local-pickup-only scope boundary as a deliberate trade-off that keeps the project buildable, not a missing feature
 - Explain how the two modules share tagging/search infrastructure despite being conceptually different
- **Hands-on skill:**
 - Actually test the request flow with two different user accounts — does User A get blocked from claiming their own listing twice, does User B get notified when it's taken

Vigil

- **Stack:** Python, Docker SDK, LangGraph, Pytest, PostgreSQL
- **What it is:** A sandbox that runs AI agent tool-calls in isolated containers, plus a harness that scores whether an agent's output is actually correct.
- **To build it:**
 - Docker SDK for Python — creating, running, and tearing down containers from code
 - Resource limits — capping CPU/memory/execution time per container
 - Pytest as a deterministic pass/fail runner — running an agent against known-correct tasks

- Basic anomaly detection — flagging excessive repeated calls by count/time (scoped honestly, not claiming to catch everything)
- **To deploy it:**
 - Different from the rest — this is an internal tool, not a user-facing product
 - "Deployment" mostly means running it locally via Docker, since it needs direct access to the Docker daemon to spin up its own sandboxed containers
 - CI/CD: still worth running its own test suite (pytest) inside GitHub Actions on every push, even without a production deploy step
 - Talking point: giving a deployed service access to the Docker socket is itself a real security consideration (a compromised app could then control the host's containers) — exactly what Vigil itself is designed to guard against
- **To explain it:**
 - Walk through the sandbox lifecycle: provision → execute → monitor → tear down
 - Why deterministic evaluation matters specifically for AI agents, since output isn't guaranteed identical run to run
 - Be precise about what the anomaly detection actually catches, and equally clear about what it doesn't — that precision is itself a good signal
- **Hands-on skill:**
 - Make sure a container actually gets cleaned up even when the process inside it crashes — a real, non-trivial thing to get right, not a one-liner

Aegis

- **Stack:** TypeScript, Playwright
- **What it is:** Automated tests that click through Trajectory the way a real user would, and check API responses directly.
- **To build it:**
 - Playwright basics: locators, assertions, running headless
 - Page Object Model (POM) — one class per page with methods like `.login()`, instead of tests touching raw selectors directly
 - Basic API assertions: status codes, response shape
- **To deploy it:**
 - No live service to deploy — this project is the CI/CD pipeline's Test stage for Trajectory, not something deployed on its own

- Runs inside GitHub Actions on every push, using a headless browser inside a Docker container provided by the CI runner
- The suite needs Trajectory running as its target — locally via Docker Compose, or against a deployed staging environment
- **To explain it:**
 - Why POM matters — when the UI changes, you update one page object instead of every test touching that page
 - What your suite actually covers: login, creating an application, changing status, uploading a resume
- **Hands-on skill:**
 - Actually watch a test fail, read the Playwright trace/screenshot output, and fix the real cause instead of just re-running it and hoping



Pulse

- **Stack:** k6
- **What it is:** Load testing Trajectory and Phoenix to see how they behave under many concurrent users.
- **To build it:**
 - k6 basics: virtual users, ramping stages
 - Load testing (normal traffic) vs. stress testing (finding the breaking point)
 - Why p95/p99 latency matters more than average — averages hide the fact that some users are having a bad time even when things look fine on average
- **To deploy it:**
 - Also runs via GitHub Actions, same CI pattern as Aegis — the load test itself is another stage in the same pipeline idea
 - The k6 script is lightweight and runs inside the CI job
 - Sends load against whatever URL you point it at — local Docker Compose setup, or a real deployed AWS environment for the most realistic numbers
- **To explain it:**
 - Walk through what you actually found — at what load did things start failing, and why (thread pool exhausted? rate limit hit? DB connections ran out?)
- **Hands-on skill:**

- Interpret the actual k6/Grafana output under real load, not just a static example screenshot — run it against your own app and see a real number

Warden

- **Stack:** OWASP ZAP, Python
- **What it is:** Security testing against Trajectory and Phoenix — automated scans plus hand-written checks for access-control bugs scanners miss.
- **To build it:**
 - OWASP Top 10 basics — SQL injection, XSS, broken access control, at a conceptual level
 - Running an automated ZAP scan
 - Writing a custom test: can User A read/modify User B's data? (Automated scanners don't reliably catch this class of bug — needs deliberate, hand-written test design)
 - Basic input fuzzing — malformed/oversized payloads
- **To deploy it:**
 - ZAP runs as a Docker container (the official ZAP image)
 - Either locally against a Docker Compose setup, or inside GitHub Actions for automated scanning on a schedule or on push — same CI pattern as Aegis and Pulse
 - Together, Aegis, Pulse, and Warden form the full Test stage of the pipeline: functional, performance, and security checks all gating a deploy before it goes out
- **To explain it:**
 - Pick one vulnerability class (broken access control is the strongest choice) and explain how you specifically tested for it
 - Be clear about what automated scanning catches well versus what needs hand-written tests — that distinction is the whole reason this project exists alongside just running ZAP alone
- **Hands-on skill:**
 - Judge a real vulnerability vs. a false positive in ZAP's output — this specifically requires your own judgment, the tool just flags candidates

Section 4: Database Fundamentals

Every backend project here runs on PostgreSQL, and this is one of the most likely places to get asked a "sounds basic, easy to blank on" question — worth being deliberate about it rather than trusting the ORM blindly.

- **Core SQL, hands-on:**
 - `SELECT` , `WHERE` , `GROUP BY` , `ORDER BY` — written by hand at least once, not only through an ORM
 - `JOIN` types — specifically `INNER` (only matching rows) vs `LEFT` (all rows from the left table, matched or not) — a very common interview question on its own
 - Primary Keys vs Foreign Keys — what each actually enforces at the database level
- **Schema design:**
 - Normalization, briefly — why data gets split across related tables (Applications, Resumes, Career Profiles in Trajectory) instead of one giant table with repeated data
 - Relationships: one-to-many, many-to-many — and how each is actually represented with foreign keys / join tables
- **Reliability:**
 - ACID, one line per letter — Atomicity (all-or-nothing), Consistency (valid state to valid state), Isolation (concurrent transactions don't interfere), Durability (once committed, it's saved)
 - Transactions — wrapping multiple operations into one atomic unit; tie this directly to Trajectory: a status update and its corresponding history-log insert should succeed or fail together, not one without the other
- **Performance (the part most likely to actually get demonstrated):**
 - Indexes — what one is, why it speeds up reads (avoids scanning every row) at a small cost to writes (the index itself has to be updated) — connects directly to anything you benchmark with Pulse
 - Connections — a database connection isn't free to open; understand why you don't open a new one per query in a loop, and that this is exactly what connection pooling exists to solve
- **Relational vs. Non-relational, conceptually:**
 - Phoenix uses a vector store (`pgvector` / `FAISS`) alongside regular PostgreSQL tables
 - Be able to say in one sentence why: relational tables are great for structured, related data (applications, users), but similarity search over embeddings needs a data structure built specifically for that job
- **Hands-on skill:**
 - Actually query the database directly — using `psql` or a GUI tool like DBeaver/pgAdmin — instead of only ever seeing data through your application's UI. Seeing the raw rows

at least once catches a surprising number of "wait, why does the data look like that" bugs.

Section 5: General Non-Scriptable Skills

These apply across every project, not tied to one specifically:

- **.env discipline** — never commit secrets, always `.gitignore` your env file
- **Git, hands-on** — actually resolve a merge conflict once, commit through a simple branch flow (`feature` → `main`), don't just read about how `git merge` works
- **Basic Linux shell** — `cd`, `ls`, `cat`, `grep` — enough to navigate inside a Docker container via `docker exec -it <container> /bin/sh`
- **Reading a real stack trace** — finding the actual originating line, not just the first line of the error message

Section 6: Fresher-Level Interview Prep — The Fundamentals That Get Asked First

These are commonly asked before anyone gets to your projects. Know these cold.

- **Exceptions** (a very common early question)
 - Checked vs unchecked exceptions in Java
 - `try / catch / finally` — and the gotcha: a return inside `finally` silently overrides a return in `try`
 - Custom exception classes — when you'd make one
 - Python equivalent: `try / except / else / finally`
- **Classic "gotcha" questions**
 - `==` vs `.equals()` in Java (reference equality vs value equality)
 - Java is always pass-by-value, even for objects (the reference itself gets copied)
 - `static` vs instance members
- **REST & HTTP basics**
 - HTTP verbs: `GET / POST / PUT / DELETE` and which ones are idempotent (same result if repeated)
 - Status code categories: 2xx success, 4xx client error, 5xx server error — and specifically 401 (not logged in) vs 403 (logged in, not allowed)
 - What "stateless" means for a REST API and for JWT specifically

- **OOP fundamentals**

- Encapsulation, Inheritance, Polymorphism, Abstraction — one clear example of each, ready to say out loud
- Composition vs Inheritance — favoring "has-a" over "is-a" when it gives more flexibility

- **Git basics**

- What a branch actually is (just a pointer to a commit)
- `git revert` (safe, makes a new commit) vs `git reset` (rewrites history, riskier)

Once fresher-level rounds are cleared and projects are further along, move to the Master Handbook for deeper coverage — distributed systems concepts, JVM internals, HLD/LLD, and the full System Breakage Drills.

Section 7: Java Language Essentials

This section covers the core language mechanics that interviewers expect a Java fresher to know cold. Focus on *why* these structures exist, *how* they behave under the hood, and *when* to use which.

1. Java Collections Framework (JCF)

- **The Hierarchy:** Core interfaces are `List` (ordered, allows duplicates), `Set` (unordered, unique elements), `Queue` (FIFO order), and `Map` (key-value pairs, separate hierarchy).
- **ArrayList vs. LinkedList:**
 - `ArrayList`: Backed by a dynamically resizing array. $O(1)$ random access, but $O(N)$ insertion/deletion in the worst case (due to array copying). Use by default for read-heavy operations.
 - `LinkedList`: Backed by a doubly-linked list. $O(N)$ random access, but $O(1)$ insertion/deletion if you already have the node reference. Higher memory overhead due to node pointers.
- **HashMap vs. TreeMap vs. LinkedHashMap:**
 - `HashMap`: Hash table-based. $O(1)$ average time complexity for `get` and `put`. Unordered.
 - `TreeMap`: Red-Black tree-based. $O(\log N)$ operations. Sorted by natural ordering or custom `Comparator`.

- `LinkedHashMap` : Maintains insertion order using a doubly-linked list running through the hash buckets. $O(1)$ lookup with predictable iteration order.
- **HashSet vs. TreeSet:**
 - `HashSet` is backed by a `HashMap` (values are dummy objects). $O(1)$ lookup, unordered.
 - `TreeSet` is backed by a `TreeMap` . $O(\log N)$ lookup, sorted.

2. Modern Java (Streams, Lambdas, Optional)

- **Lambda Expressions:** Syntax to implement functional interfaces (interfaces with a single abstract method, e.g., `Runnable` , `Callable` , `Comparator`). Enables passing behavior as parameters.
 - Example: `list.sort((a, b) -> a.compareTo(b));`
- **Streams API:** A sequence of elements supporting sequential and parallel aggregate operations. Streams do not store data; they operate on a source.
 - **Intermediate Operations:** Lazy evaluation. Returns a new stream. Examples: `filter()` , `map()` , `flatMap()` , `sorted()` .
 - **Terminal Operations:** Triggers stream processing. Returns a non-stream result (or void). Examples: `collect()` , `reduce()` , `forEach()` , `findFirst()` .
 - *Gotcha:* A stream cannot be reused once a terminal operation is called.
- **Optional:** A container object which may or may not contain a non-null value. Designed to replace `null` returns and prevent `NullPointerException` .
 - Use `orElse(defaultVal)` (always evaluates default value) or `orElseGet(() -> getVal())` (lazy evaluation of default value).
 - Use `map()` and `flatMap()` to chain operations without explicit null checks.

3. Multithreading Basics

- **Process vs. Thread:** A process is an executing instance of an application with its own memory space. A thread is the smallest unit of execution within a process; threads share the process's memory space (Heap) but have their own Stack.
- **Creating Threads:**
 - Extend `Thread` class or implement `Runnable` interface. Implementing `Runnable` is preferred because Java only supports single inheritance.

- Implement `Callable<T>` if you need the thread to return a value (`Future<T>`) or throw checked exceptions.
- **Thread Safety:**
 - `synchronized` : Locks an object or class to ensure only one thread executes the block at a time.
 - `volatile` : Ensures visibility of changes to variables across threads by forcing reads/writes directly to main memory (disables CPU caching).
 - `ReentrantLock` : Offers advanced lock features like fairness, interruptible lock attempts, and try-locking.
 - `AtomicVariables` (e.g., `AtomicInteger`): Use low-level CPU instructions (Compare-And-Swap) to perform thread-safe atomic operations without locking.
- **ExecutorService & Thread Pools:** Thread creation is expensive. A thread pool reuses a fixed number of threads to execute tasks.
 - `Executors.newFixedThreadPool(n)` : Limits concurrent threads.
 - `Executors.newCachedThreadPool()` : Spawns threads as needed, reuses idle ones.
 - *Gotcha:* Be ready to explain what happens to tasks when the queue fills up (`RejectedExecutionException`).

4. Generics

- **Why Generics?** Provides compile-time type safety. Catches type mismatches at compile time rather than throwing a `ClassCastException` at runtime. Eliminates the need for manual casting.
- **Wildcards & PECS Rule:**
 - **Producer Extends:** Use `? extends T` when you want to read items of type `T` from a collection (covariance).
 - **Consumer Super:** Use `? super T` when you want to write items of type `T` into a collection (contravariance).
 - Example: `public static <T> void copy(List<? super T> dest, List<? extends T> src)`

5. Memory Fundamentals (Stack vs. Heap, String Pool, GC)

- **Stack vs. Heap:**
 - **Stack:** Stores local variables and method call frames. Fast allocation, private to each thread, automatic cleanup when the method returns.

- **Heap:** Stores all objects and instance variables. Shared across threads, allocation requires GC, garbage collected when objects are no longer reachable.
- **String Constant Pool:** A special memory region in the Heap that stores String literals.
 - Strings are immutable in Java for security, caching, and synchronization.
 - `String s1 = "Java"; String s2 = "Java";` (`s1 == s2` is true - refers to same object in String Pool).
 - `String s3 = new String("Java");` (`s1 == s3` is false - s3 is a new object on the heap outside the pool).
 - `s3.intern()` returns the canonical representation from the String Pool.
- **Garbage Collection (GC):** Automatic memory management process that identifies and deletes unreachable objects on the heap.
 - **Generational Hypothesis:** Most objects die young.
 - **Heap Divisions:** Young Generation (Eden, Survivor spaces S0/S1) where new objects are created, Old (Tenured) Generation for long-lived objects, and Metaspace (method metadata, outside JVM heap).
 - **GC Steps:** Mark (identify live objects), Sweep (clear dead objects), Compact (reorganize memory to avoid fragmentation).

6. Enums and Comparable/Comparator

- **Enums:** Typesafe representation of a fixed set of constants. Enums can have fields, constructors (private), and methods.
- **Comparable vs. Comparator:**
 - `Comparable<T>` : Implements natural ordering. Must override `int compareTo(T o)` . The class itself implements it.
 - `Comparator<T>` : Implements custom/multiple sorting strategies. Must override `int compare(T o1, T o2)` . Typically implemented as a separate class, anonymous class, or lambda expression.